

瓶颈链路队列管理对实时游戏流量尾延迟的影响：Tail-drop、CoDel 与 L4S 双队列方案的仿真比较

Impact of Bottleneck Queue Management on Tail Latency of Real-Time Game Traffic: A Simulation Comparison of Tail-drop, CoDel, and L4S Dual-Queue Schemes

Fluentrum, copycat666^{1,2*}, doro^{1,2}

1. EigenLife 机器学习实验室，东北电力大学，吉林；2. 《电子游戏科学与技术》编辑委员会，吉林，
submission.joest.cn@outlook.com

*: Corresponding Author

摘要：实时对战类游戏的客户端以固定周期发送小尺寸数据报，其网络需求集中在排队时延能否获得稳定的上界。当瓶颈链路上同时存在下载、视频等长流时，缓冲区会被持续填满，排队时延成为影响游戏体验的主要网络因素。本文建立包级的离散事件瓶颈链路仿真模型，以带抖动的周期小包刻画游戏流量，以若干响应式长流和恒速非响应式流量刻画背景流量，在同一组参数下比较 Tail-drop、RED、CoDel 与 L4S 双队列四类队列管理方案。仿真覆盖六档非响应式负载、五档背景流数量与五档缓冲区容量，每档重复五组随机种子。结果表明：在 150 kB 缓冲区与中等拥塞下，Tail-drop 的中位排队时延约为 111 至 118 毫秒，并随缓冲区容量近似线性增长；RED 与 CoDel 分别将其压缩到约 31 至 59 毫秒与 27 至 88 毫秒，但两者的 99 分位时延随非响应式负载占比上升而显著恶化；L4S 双队列把游戏包的中位与 99 分位排队时延稳定在约 0.69 毫秒与 1.27 毫秒，且对缓冲区容量、背景流数量与负载结构均不敏感。四种方案的链路利用率都接近满载。结果可为接入网低时延队列管理的选型提供参考。

关键词：实时游戏；排队时延；尾延迟；主动队列管理；低延迟低损耗可扩展吞吐；拥塞控制

Abstract. Clients of real-time competitive games send small datagrams at a fixed cadence, so their network requirement is dominated by whether queueing delay has a stable upper bound. When long-lived flows such as downloads and video streams share the bottleneck link, the link buffer is kept nearly full and queueing delay becomes the main network factor affecting the gaming experience. This paper builds a packet-level discrete-event model of a single bottleneck link, in which game traffic is represented by a periodic small-packet flow with jitter and background traffic is represented by several responsive long-lived flows plus a constant-rate unresponsive flow. Under one common set of parameters, four queue management schemes are compared: Tail-drop, RED, CoDel, and the L4S dual-queue coupled AQM. The experiments cover six levels of unresponsive load, five levels of responsive background flow count, and five buffer sizes, with five random seeds per configuration. At a 150 kB buffer and moderate congestion, the median queueing delay of game packets under Tail-drop is about 111 to 118 ms and grows almost linearly with buffer size; RED and CoDel compress it to about 31 to 59 ms and 27 to 88 ms respectively, but their 99th-percentile delay degrades markedly as the unresponsive load grows; the L4S dual-queue scheme keeps the median and 99th-percentile queueing delay of game packets at about 0.69 ms and 1.27 ms, and is insensitive to buffer size, flow count, and load structure. All four schemes keep link utilization close to full, which indicates that very low queueing delay can be obtained together with near-saturated throughput.

Keywords: Real-time gaming; Queueing delay; Tail latency; Active queue management; L4S; Congestion control

一、引言

网络化实时对战类游戏（多人在线战术竞技、第一人称射击等）的客户端通常在每一个网络节拍内向服务器上报一次玩家输入与本地状态，上报周期大致对应每秒 20 至 60 个数据报，单个数据报的有效载荷在几十字节到二百字节之间，因此单个客户端的上行速率一般只有每秒数十千比特的量级 [1, 2]。这样的带宽需求相对于接入链路的容量可以忽略，其体验主要取决于数据包在路径上经历的时间。游戏对时延的要求体现在两个方面：端到端时延的绝对水平要低，同时其时延分布要有稳定的上界，也就是尾延迟不能长时间停留在高值。近年针对云游戏与实时通信的测量工作反复指出，造成玩家体验下降的往往是时延的长时间高值区间 [3]。

接入链路是端到端路径上最常见的瓶颈。当链路上同时存在文件下载、视频流等长流时，若链路边界的缓冲区按最大化吞吐的原则配置，队列长度会长期接近缓冲上限，所有经过该队列的数据包都要承受与缓冲区容量成正比的排队时延，这一现象通常称为缓冲膨胀（bufferbloat）[4]。对于以低速率小包为主的游戏流量，其自身发送速率不足以在资源竞争中换取低时延，因此其排队时延在很大程度上由瓶颈队列的管理策略决定。

围绕这一问题,网络侧与端系统侧都出现了新的机制。网络侧的代表是主动队列管理(active queue management, AQM)算法族,包括随机早期检测(random early detection, RED) [5]、受控延迟队列管理(controlled delay, CoDel) [6]与比例积分控制器增强(proportional integral controller enhanced, PIE) [7];端系统侧的代表是模型驱动的BBR [8]与以时延为目标的Copa [9]。这些机制都建立在显式拥塞通知(explicit congestion notification, ECN) [10]以及一批成熟拥塞控制算法的基础之上,例如数据中心TCP [11, 12]、CUBIC [13]与承载于QUIC之上的拥塞控制 [14]。近年的低延迟低损耗可扩展吞吐(low latency, low loss and scalable throughput, L4S)体系结构把两侧的改动组合起来:网络侧使用浅门限的ECN标记与双队列耦合AQM,端系统侧使用可扩展拥塞控制 [15, 16, 17]。已有评测报告, L4S可以在共享瓶颈上把排队时延的均值控制在亚毫秒级别、99分位控制在2毫秒以内 [18],相应的端主机与队列模型也已进入仿真平台 [19]。

除队列管理本身,应用侧也出现了面向尾延迟的设计。云游戏平台通过感知感知质量的码率控制来压缩带宽开销与时延 [3],实时通信中的拥塞控制开始借助物理层信息直接估计可用带宽 [20],接收端调度则被用来消除网络抖动带来的播放不稳 [21]。在游戏网络的其他方向上,多接入网络中的低时延移动性管理 [22]、虚拟现实云游戏的资源分配与视点渲染 [23, 24]、以及网络层作弊的检测 [25]都得到了讨论。

上述工作分别针对协议设计、端系统算法或特定应用,覆盖的评价场景与参数设置差异较大,因而难以直接回答一个具体的问题:在同一个瓶颈模型下,当背景流量同时包含可响应的长流与不受控的恒速流量时,不同的队列管理方案会把游戏数据包的排队时延控制在什么水平,其代价又是什么。本文以包级离散事件仿真为工具,在统一的链路容量、往返时延与流量构成下对四种方案做横向比较,覆盖负载结构、背景流数量与缓冲区容量三个维度,并同时报告时延分位数与吞吐代价。本文的工作可以概括为三点:(1)建立一个包含响应式长流与非响应式恒速流的包级瓶颈链路模型,并把游戏流量建模为带抖动的周期小包流;(2)在同一模型下实现并比较Tail-drop、RED、CoDel与L4S双队列四种队列管理方案,其中ECN标记的反应方式按各方案对应的规范设置;(3)给出时延分位数、丢包率与吞吐代价的定量结果,并说明各方案的时延对缓冲区容量的依赖关系存在本质差别。

二、相关工作

2.1. 游戏与实时应用的网络需求

实时游戏的流量特征已有较系统的刻画。测量工作指出,游戏客户端的数据报以固定周期发送、单包很小且速率很低,这类流量的服务质量主要由时延及其抖动决定 [2]。针对云游戏的大规模测量发现,即便部署了边缘服务器与专门的拥塞控制,无线局域网的最后一段链路仍然是长尾时延的主要来源,其中短时的发送干旱是直接原因 [1]。在系统设计层面,接收端释放调度被用于还原发送端的时间节奏,从而抑制抖动引起的卡顿 [21];码率控制则在保证感知质量的前提下降低带宽占用 [3]。面向下一代无线网络的工作进一步把注意力放到资源分配与渲染策略上,例如按视觉注意力分配分辨率与帧率 [23, 24]。这些工作共同说明,小包低速率交互流量的核心指标是排队时延的上界,而非平均吞吐。

2.2. 拥塞控制与队列管理的演进

队列管理的研究可以追溯到随机早期检测,它用队列长度的指数加权平均来提前丢弃或标记数据包,避免队列长期满置 [5]。ECN的引入使网络可以把拥塞状态告知发送端而不必丢弃数据包 [10],数据中心TCP进一步利用标记的比例信息构造了平滑的窗口调整规则 [11, 12]。在发送端的窗口增长律方面,CUBIC面向长肥管道改进了拥塞避免阶段 [13],并强调了新算法与Reno兼容的必要性 [26];QUIC作为用户态传输协议则为拥塞控制与ECN反馈提供了新的承载 [14]。与此同时,模型驱动的BBR通过估计瓶颈带宽与最小往返时延来控制发送速率 [8],以时延为目标的Copa则把目标时延作为显式参数 [9]。

在队列管理一侧,CoDel用数据包的驻留时间而不是队列长度作为控制变量,从而在链路速率变化时保持配置不变 [6];PIE以比例积分控制器逼近目标时延 [7]。国际互联网工程任务组对AQM的部署建议指出,拥塞信号应当尽早发出,并且算法的稳态时延目标应当以时间为单位表达 [4]。这些算法为本文的对比提供了基准。

2.3. 低延迟双队列耦合 AQM

L4S的基本判断是,仅靠网络侧或仅靠端系统侧都难以进一步压缩排空容量寻址流所产生的队列。它的做法是在两端同时改动:网络侧为L4S数据包提供一条浅缓冲的低延迟队列,并用ECN的浅门限标记产生稠密而温和的拥塞信号;端系统侧使用可扩展拥塞控制,使窗口的锯齿幅度变小、恢复时间不随速率增长 [15, 16]。为了避免可扩展流量挤占经典流量,双队列耦合AQM让经典队列的丢弃概率与低延迟队列的标记概率保持平方关系,从而在两种队列之间建立吞吐等价 [17]。实测与仿真结果表明,该结构可以把低延迟队列的平均排队时延控制在1毫秒以内,99分位控制在2毫秒以内 [18];ns-3中也已经出现包含TCP Prague与准确ECN反馈的完整实现,用于可复现的评测 [19]。

2.4. 本文的定位

已有针对 L4S 的评测多以吞吐等价性和负载适应性为主线，游戏流量通常只作为低速率交互流量的一个代表被间接提及；而面向游戏网络的测量与系统工作又往往不涉及队列管理算法的横向比较。本文把两者结合起来，以一个受控的单一瓶颈模型为平台，把游戏数据包的排队时延分位数作为主要指标，在负载结构、背景流数量与缓冲区容量三个维度上对四种队列管理方案做对照实验，并同时记录链路利用率与两类背景流量的吞吐，以便观察低时延是否以吞吐为代价。

三、仿真模型与方法

3.1. 瓶颈链路与流量构成

模型由一条瓶颈链路以及挂在它上面的三类流量源组成。链路容量记为 C （比特/秒），缓冲区容量记为 B （比特）。三类流量的数据包按到达先后进入队列，第 i 个数据包的服务时间为

$$s_i = \frac{L_i}{C}, \quad (1)$$

其中 L_i 为该数据包的总长度（比特）。数据包的排队时延定义为它离开链路队列的时刻与它进入队列的时刻之差，

$$d_i = t_i^{\text{dep}} - t_i^{\text{enq}}. \quad (2)$$

统计窗口内的链路利用率为

$$U = \frac{\sum_{i \in \mathcal{W}} L_i}{C T_{\mathcal{W}}}, \quad (3)$$

其中 $\mathcal{W}$ 表示统计窗口， $T_{\mathcal{W}}$ 为其时长。

第一类流量是游戏流量，它由游戏客户端周期性产生的状态与输入上报构成。模型把它记为速率恒定的周期小包流：以每秒 30 个数据包为基准，每个数据包长 100 字节，相邻数据包之间的间隔带有正负 10 百分比以内的随机抖动，以避免与背景流形成同步。该流量不执行拥塞控制，其发送速率不随网络状态变化。第二类流量是响应式背景流量，用来代表客户端上并发的下载或更新任务。模型使用 k 条长期存在的加性增乘性减（additive increase multiplicative decrease, AIMD）流，数据包长 1500 字节，具备 ECN 能力。其窗口按标准规则演化：在拥塞避免阶段，每收到一个确认将窗口增加 $1/\text{cwnd}$ ；在检测到丢包时，将窗口减半并令慢启动门限等于新的窗口值。第三类流量是非响应式背景流量，用来代表播放器或操作系统发起的不受控长流。模型以一个恒速源刻画它，其发送速率等于链路容量的 ρ 倍，数据包长 1500 字节，不具备 ECN 能力，因此既不响应丢包也不响应标记。

3.2. 队列管理方案

本文比较四种队列管理方案。

(1) Tail-drop 采用单一先进先出队列，仅当缓冲区剩余空间不足以容纳新到达的数据包时将其丢弃。它的控制变量是缓冲区的占满程度，因此瞬时排队时延与队列长度直接相关。

(2) RED 使用队列长度的指数加权平均 $\bar{q}$ 作为控制变量，在每个数据包到达时按

$$\bar{q} \leftarrow (1 - w) \bar{q} + w q \quad (4)$$

更新，其中 w 为平滑权重， q 为当前队列长度。当 $\bar{q}$ 落在最小门限与最大门限之间时，标记或丢弃的概率在零与最大概率之间线性插值；超过最大门限时按概率 1 处置。对具备 ECN 能力的数据包以标记代替丢弃，否则直接丢弃。

(3) CoDel 以数据包在队列中的驻留时间作为控制变量。当队头数据包的驻留时间超过目标值并持续一个控制间隔之后，算法进入丢弃状态，此后按控制间隔与状态内计数之商作为间隔依次处置数据包，计数随处置次数增加，处置频率随之上升；一旦驻留时间回落到目标值以下，算法立即退出该状态。本文取目标值为 5 毫秒、控制间隔为 100 毫秒，与规范一致 [6]。为便于与其余方案在 ECN 可用时比较，CoDel 对具备 ECN 能力的数据包以标记代替丢弃。

(4) DualQ 采用两条队列。低延迟队列容纳游戏流量，使用独立的小缓冲区（4 毫秒对应的比特数）与 1 毫秒的标记门限，驻留时间超过门限即标记；经典队列容纳全部背景流量，使用与 CoDel 相同的目标值与控制间隔，并且其丢弃频率与低延迟队列的标记率耦合，标记率越高则经典队列处置越频繁，用来避免经典流量被长期压制。调度器优先服务低延迟队列。端系统侧的反应方式随方案变化：在 Tail-drop 与 RED、CoDel 场景下，具备

ECN 能力的发送端沿用经典 ECN 的反应方式，即把标记视为丢包并将窗口减半 [10]；在 DualQ 场景下，采用可扩展反应方式，按标记比例温和调整窗口，

$$\alpha \leftarrow (1 - g)\alpha + gF, \quad \text{cwnd} \leftarrow \left(1 - \frac{\alpha}{2}\right)\text{cwnd}, \quad (5)$$

其中 F 为本次确认的标记指示， g 为平滑系数，取 $1/16$ 。

表 1 汇总了仿真参数。仿真器为离散事件驱动，事件包括数据包到达、链路发送完成与确认到达，全部逻辑在单个进程中顺序执行，因而是可复现的。主流程见算法 1。

表 1: 仿真参数

参数	取值
瓶颈链路容量 C	10 Mbit/s
基准往返时延	20 ms
缓冲区容量 B	150 kB（另做 50 至 600 kB 的扫描）
游戏流量	30 包/s, 100 字节/包, 间隔抖动正负 10 百分比
响应式背景流	4 条（另做 1 至 12 条的扫描）
响应式流数据包长	1500 字节
非响应式背景流量	链路容量的 0 至 0.9 倍
非响应式流数据包长	1500 字节
RED 门限与参数	最小门限为缓冲区的 20 百分比, 最大门限为 60 百分比, 最大概率 0.1, 权重 0.002
CoDel 目标与间隔	5 ms 与 100 ms
DualQ 低延迟队列	缓冲 4 ms, 标记门限 1 ms
单次仿真时长与统计窗口	60 s 与 50 s
随机种子	11、23、37、53、71

Algorithm 1 瓶颈链路离散事件仿真的主循环

Require: 链路容量 C , 缓冲区容量 B , 队列管理策略, 仿真时长 T

Ensure: 游戏包排队时延样本集, 利用率与吞吐统计

```

1: 初始化事件队列, 注册游戏源与各背景流的首个发送事件
2: while 事件队列非空 do
3:   取出时间最早的事件  $(t, type, payload)$ 
4:   if  $t > T$  then
5:     跳出循环
6:   end if
7:   if  $type = \text{到达}$  then
8:     按策略在队列入队处决定接受、标记或丢弃
9:     if 链路空闲 then
10:      取出队首数据包, 按式 (1) 调度发送完成事件
11:    end if
12:    else if  $type = \text{发送完成}$  then
13:      若为游戏包则按式 (2) 记录排队时延
14:      按式 (3) 累计已发送比特数, 并继续调度下一个数据包
15:    else
16:      按方案对应的规则更新发送端窗口, 并尝试发送新数据包
17:    end if
18:    统一结算本轮产生的丢包, 更新相应发送端的窗口
19:  end while
20: return 统计结果

```

3.3. 评价指标与实验设计

主要指标是游戏数据包排队时延的分位数，取中位数与 95、99 分位，并记录最大值；辅助指标是游戏包的丢包率、链路利用率、响应式背景流量与非响应式背景流量各自获得的吞吐。为便于观察负载结构的影响，实验分为三组。第一组固定背景流数量与缓冲区容量，把非响应式流量占比从零逐步提高到链路容量的 0.9 倍。第二组固定非响应式流量占比为零，把响应式背景流数量从 1 条增加到 12 条。第三组固定背景流数量与非响应式流量占比为 0.5 倍，把缓冲区容量从 50 kB 增加到 600 kB。每组中的每个配置都使用五组随机种子重复运行，报告其算术平均。统计窗口为 10 秒之后到 60 秒结束，以避免启动暂态。

四、结果与分析

4.1. 非响应式背景流量的影响

图 1 给出三组指标随非响应式流量占比变化的情况，表 2 列出其中三档的代表值。Tail-drop 的中位排队时延几乎不随负载结构变化，始终位于 111 至 118 毫秒之间，这是因为只要缓冲区内还能容纳数据包，队列就会一直保持接近满置的状态，其排队时延由缓冲区容量与链路速率之比决定，即 B/C 。在本组参数下该比值为 120 毫秒，与实测的中位数一致。

RED 与 CoDel 都能在非响应式流量占比不高时明显压低时延。占比为零时，两者的中位时延分别为 31.1 与 26.5 毫秒，不到 Tail-drop 的三分之一。随着非响应式占比上升，两者的优势逐步缩小：占比为 0.8 时，RED 的中位时延升到 51.6 毫秒，CoDel 升到 73.6 毫秒；占比为 0.9 时，CoDel 的 99 分位回到 119.2 毫秒，与 Tail-drop 基本持平。这一退化的原因在于，两者都依赖发送端对拥塞信号做出退让，而恒速流量不会调整自身速率，因此随着不受控流量的比例增大，队列的控制权重新回到缓冲区容量手中。

DualQ 的表现对负载结构不敏感。在整个扫描范围内，游戏包的中位排队时延保持在 0.67 至 0.69 毫秒之间，99 分位不超过 1.27 毫秒，最大值不超过 1.28 毫秒。由于游戏流量被隔离在浅缓冲的低延迟队列中，其排队时延与经典队列的长度基本无关；只有在低延迟队列自身的短暂突发时才出现毫秒级的瞬时值。该结果与规范中给出的目标一致，即低延迟队列的平均排队时延应低于 1 毫秒、99 分位不高于 2 毫秒 [17]。

从吞吐的角度看，四种方案的链路利用率都接近满载，稳定在 99.9 百分比以上，说明低时延的获得没有牺牲链路容量。表 2 同时给出两类背景流量的吞吐分配。在非响应式占比为 0.8 时，响应式背景流获得的吞吐依次为 2.21、2.87、3.77 与 4.81 Mbit/s，对应非响应式流量为 7.77、7.10、6.21 与 5.17 Mbit/s。可以看到，队列管理越主动，可响应流量获得的比例越高，而恒速的非响应式流量相应减少；四种方案的总吞吐都保持在 9.98 Mbit/s 左右，与链路容量相差不超过 0.2 百分比。

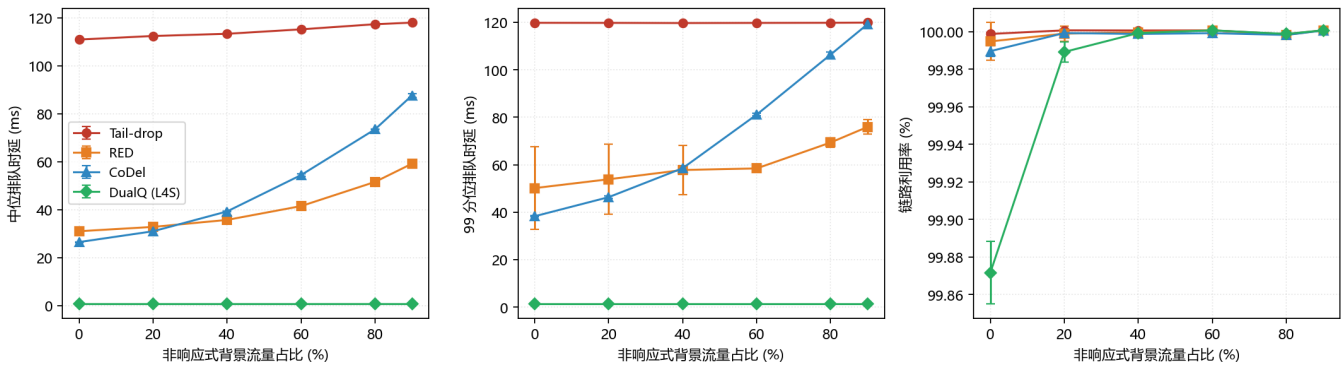


图 1: 非响应式背景流量占比对游戏包排队时延与链路利用率的影响（五组种子的均值，误差棒为标准差）

4.2. 缓冲区容量的影响

图 2 与表 3 给出缓冲区容量从 50 kB 增加到 600 kB 的结果，此时非响应式流量占比固定为 0.5。四种方案的差别在这一维度上表现得最清楚。

Tail-drop 的中位排队时延几乎与缓冲区容量成正比：50 kB 时为 38.5 毫秒，150 kB 时为 114.3 毫秒，600 kB 时为 453.7 毫秒。把中位时延与 B/C 相比，其比值稳定在 0.94 至 0.96 之间，说明队列在统计意义上长期处于满置附近。RED 的中位时延同样随缓冲区容量增长，从 19.1 毫秒升到 112.1 毫秒，增速慢于 Tail-drop，但其 99 分位在 600 kB 时达到 267.0 毫秒，已经失去对尾延迟的控制。CoDel 的表现介于两者之间并呈现出不同的形态：在 100 kB 及以上，其中位时延稳定在 46 至 47 毫秒附近，99 分位稳定在 67 至 72 毫秒附近，基本不随缓冲区容

表 2: 不同非响应式负载下的游戏包排队时延（中位/99 分位, ms）与吞吐分配 (Mbit/s)

方案	占比	中位	99 分位	响应式吞吐	非响应式吞吐	利用率
Tail-drop	0	111.0	119.8	9.98	0.00	1.000
Tail-drop	0.4	113.4	119.7	6.01	3.96	1.000
Tail-drop	0.8	117.4	119.8	2.21	7.77	1.000
RED	0	31.1	50.1	9.98	0.00	1.000
RED	0.4	35.8	57.8	6.16	3.82	1.000
RED	0.8	51.6	69.4	2.87	7.10	1.000
CoDel	0	26.5	38.3	9.98	0.00	1.000
CoDel	0.4	39.3	58.5	6.45	3.52	1.000
CoDel	0.8	73.6	106.4	3.77	6.21	1.000
DualQ	0	0.68	1.27	9.96	0.00	0.999
DualQ	0.4	0.68	1.27	7.25	2.73	1.000
DualQ	0.8	0.68	1.27	4.81	5.17	1.000

量变化。这说明 CoDel 确实把控制变量从队列长度换成了时间，从而把稳态时延与缓冲区容量解耦。需要说明的是，其稳态时延明显高于 5 毫秒的目标值，原因在于控制律要求驻留时间持续超过目标一个完整控制间隔之后才动作，而在只有少量长流的场景中，退让之后队列会在下一个动作到来之前重新被推高，形成幅度较大的锯齿。

DualQ 的中位时延在全部缓冲区容量下都保持在 0.68 至 0.69 毫秒，99 分位保持在 1.27 毫秒以内，与缓冲区容量完全无关。这与低延迟队列使用独立的 4 毫秒小缓冲直接相关：经典队列的增长不会传递到游戏流量所在的队列上。

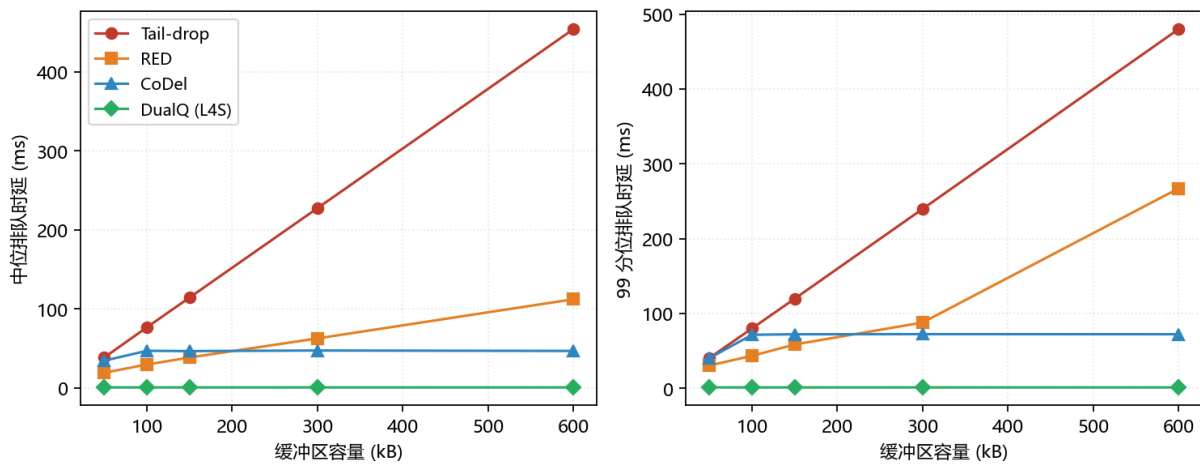


图 2: 缓冲区容量对游戏包排队时延的影响（非响应式流量占比固定为 0.5）

表 3: 不同缓冲区容量下的游戏包排队时延（中位/99 分位, ms）

方案	50 kB	100 kB	150 kB	600 kB
Tail-drop	38.5 / 40.5	76.6 / 80.1	114.3 / 119.7	453.7 / 479.8
RED	19.1 / 30.6	29.7 / 43.4	38.5 / 58.6	112.1 / 267.0
CoDel	34.5 / 40.4	47.0 / 71.6	46.6 / 72.2	46.8 / 72.3
DualQ	0.69 / 1.27	0.68 / 1.27	0.69 / 1.27	0.69 / 1.27

4.3. 背景流数量的影响

表 4 给出响应式背景流数量从 1 条增加到 12 条、非响应式流量占比为零时的结果。Tail-drop 的中位时延从 93.6 毫秒单调升到 117.0 毫秒，与队列在更多竞争流下更容易保持满置相符。RED 的中位时延从 28.0 毫秒升到 47.8 毫秒，同时在背景流数量较少时出现吞吐损失：1 条流时利用率为 0.935，2 条流时为 0.976。其原因在于

RED 以队列长度的平滑平均为控制变量，在流数少、队列波动慢的情况下标记过于提前，使发送端窗口长期低于填满链路所需的值。CoDel 的变化不单调，1 条流时为 9.3 毫秒，4 条流时升到 26.5 毫秒，8 条及以上又回落到 18 毫秒附近；流数增加使驻留时间的估计更稳定，控制动作也更及时。DualQ 的中位时延在全部流数下都保持在 0.67 至 0.69 毫秒。

4.4. 时延分布与时间行为

图 3 给出非响应式占比为零与 0.6 两种情况下游戏包排队时延的互补累积分布。占比为零时，CoDel 与 RED 的分布在 40 至 50 毫秒处降到千分之一量级，Tail-drop 则要到 115 至 120 毫秒才急剧下降，两条曲线的横坐标相差约三倍。占比升到 0.6 后，RED 与 CoDel 的分布整体右移，而 Tail-drop 与 DualQ 的分布位置基本不变，说明前两者对非响应式流量敏感，后两者分别因为控制权在缓冲区容量和因为队列隔离而不敏感。

图 4 给出单个种子下四种方案的排队时延时间序列。Tail-drop 的曲线长时间贴在 100 至 120 毫秒的区间内小幅波动；RED 的曲线在 25 至 50 毫秒之间波动，并在启动阶段出现过接近 100 毫秒的瞬时值；CoDel 的曲线呈周期性锯齿，峰值可达 60 至 70 毫秒，落点接近 10 毫秒；DualQ 的曲线密集分布在 0.2 至 1.3 毫秒之间，没有任何持续时间较长的高值区间。这一对比说明，衡量交互流量的时延不能只看平均值：CoDel 与 RED 在均值意义上的改善是明显的，但其尾部的锯齿仍会给玩家带来可感知的间歇性延迟。

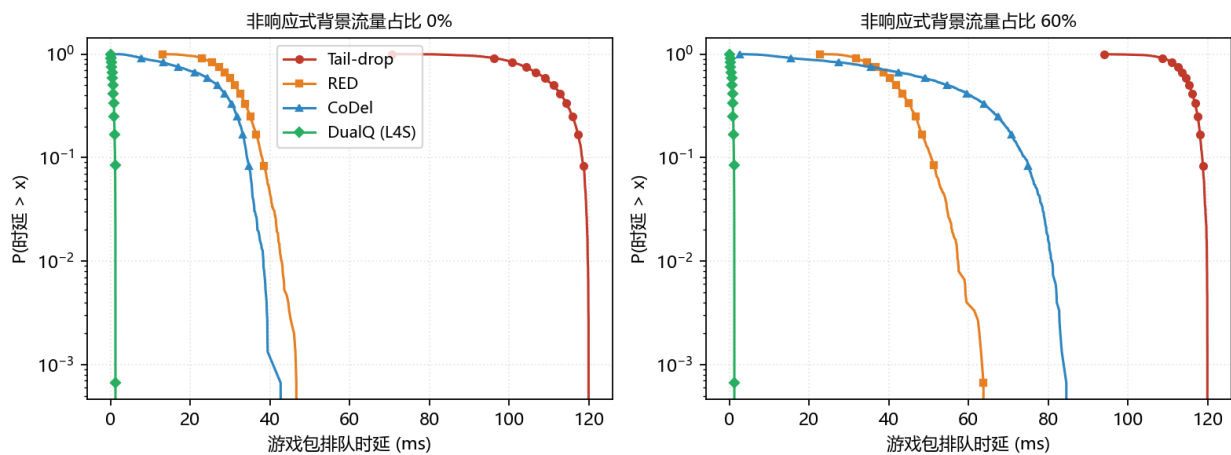


图 3：游戏包排队时延的互补累积分布（左：非响应式占比为零；右：占比 0.6）

表 4：不同响应式背景流数量下的游戏包排队时延（中位/99 分位，ms）与利用率

方案	1 条流	2 条流	8 条流	12 条流
Tail-drop	93.6 / 119.5	107.1 / 119.7	115.7 / 119.8	117.0 / 119.9
RED	28.0 / 81.2	29.2 / 88.7	39.4 / 50.0	47.8 / 57.9
CoDel	9.3 / 15.3	14.1 / 23.1	8.7 / 9.3	18.3 / 18.9
DualQ	0.69 / 1.27	0.69 / 1.27	0.68 / 1.27	0.67 / 1.26
RED 利用率	0.935	0.976	1.000	1.000

4.5. 讨论

三组实验指向同一个结论：决定小包交互流量排队时延上界的因素是队列管理是否把控制变量从队列长度改成了时间，以及是否存在与本流量无关的独立队列。Tail-drop 把全部流量放在同一个先进先出队列里，其排队时延由缓冲区容量唯一决定；RED 仍然以队列长度为依据，只是把丢弃提前，因此缓解但不解除这一依赖；CoDel 改用驻留时间，把稳态时延与缓冲区容量解耦，但仍受同一队列中其他流量的影响；DualQ 在时间控制之外再加一层队列隔离与端系统侧的可扩展反应，使游戏流量的排队时延同时与缓冲区容量、背景流数量和负载结构无关。

本文的结论建立在若干简化之上，需要在解读时加以注意。模型只包含一条瓶颈链路，没有建模无线接入段、路径上的其他排队点以及多瓶颈下的相互作用；背景流量用理想化的 AIMD 窗口机制刻画，没有区分快速重传与超时重传，也没有引入多流之间的同步效应；队列管理算法在入队处执行，而部分实现选择在出队处执

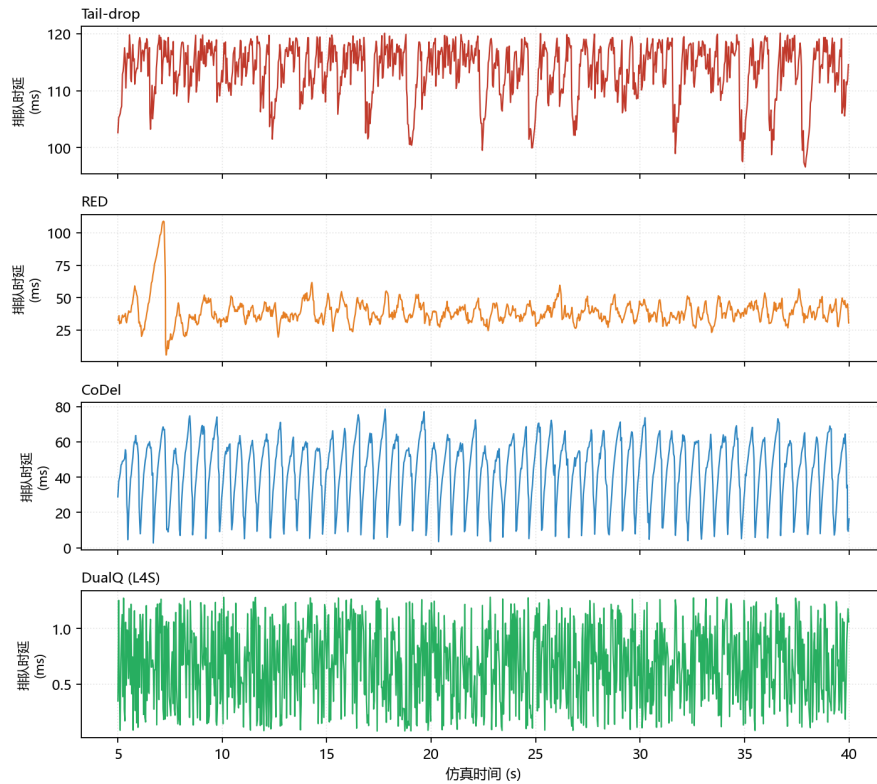


图 4: 四种队列管理方案下游戏包排队时延的时间序列（非响应式占比 0.5）

行，这会给控制回路引入不同的反馈延迟；游戏流量被建模为单一恒定速率的周期源，未涉及应用层的回滚、重传与状态同步逻辑；此外，本文没有对端系统侧算法的收敛特性做敏感性分析。这些简化会影响绝对数值，但不太可能改变方向性结论，因为队列隔离与时间控制这两个机制的作用并不依赖于上述细节。后续工作可以在真实接入链路上做端到端验证，并把游戏应用层的冗余传输与本章的队列管理结果联合考虑。

五、结论

本文用包级离散事件仿真模型比较了 Tail-drop、RED、CoDel 与 L4S 双队列四种队列管理方案对实时游戏流量排队时延的影响，覆盖非响应式负载、响应式背景流数量与缓冲区容量三个维度。在 10 Mbit/s 瓶颈链路、20 毫秒基准往返时延与 150 kB 缓冲区的基准配置下，Tail-drop 的中位排队时延为 111 至 118 毫秒，RED 与 CoDel 分别为 31 至 59 毫秒与 27 至 88 毫秒，L4S 双队列为 0.67 至 0.69 毫秒，四种方案的 99 分位时延差别更大。缓冲区容量从 50 kB 增加到 600 kB 时，Tail-drop 的中位时延近似线性增长约十二倍，RED 增长约六倍，CoDel 基本不变，L4S 双队列完全不变。四种方案的链路利用率都接近满载，低时延的获得没有以牺牲链路容量为代价，其代价主要体现在两类背景流量之间的吞吐分配上。上述结果表明，在瓶颈设备上引入时间驱动的队列管理与流量隔离，可以把游戏这类低速率小包流量的排队时延从百毫秒量级压缩到亚毫秒量级。

参考文献

- [1] Fengqian Guo et al. *BLADE: Adaptive Wi-Fi Contention Control for Next-Generation Real-Time Communication*. arXiv:2603.16119. 2026.
- [2] Francesco Vitale et al. *Network Traffic Analysis with Process Mining: The UPSIDE Case Study*. arXiv:2512.23718. 2025.
- [3] Houde Qian et al. *SAIL: Perceptual Quality-Aware Rate Control for Cloud Gaming*. arXiv:2607.11231. 2026.
- [4] Fred Baker and Gorrry Fairhurst. *IETF Recommendations Regarding Active Queue Management*. RFC 7567. RFC Editor, 2015.

- [5] Sally Floyd and Van Jacobson. “Random Early Detection Gateways for Congestion Avoidance”. In: *IEEE/ACM Transactions on Networking* 1.4 (1993), pp. 397–413.
- [6] Kathleen Nichols et al. *Controlled Delay Active Queue Management*. RFC 8289. RFC Editor, 2018.
- [7] Rong Pan et al. *Proportional Integral Controller Enhanced (PIE): A Lightweight Control Scheme to Address the Bufferbloat Problem*. RFC 8033. RFC Editor, 2017.
- [8] Neal Cardwell et al. “BBR: Congestion-Based Congestion Control”. In: *Communications of the ACM* 60.2 (2017), pp. 58–66.
- [9] Venkat Arun and Hari Balakrishnan. “Copa: Practical Delay-Based Congestion Control for the Internet”. In: *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 2018, pp. 329–342.
- [10] Kadangode Ramakrishnan, Sally Floyd, and David Black. *The Addition of Explicit Congestion Notification (ECN) to IP*. RFC 3168. RFC Editor, 2001.
- [11] Stephen Bensley et al. *Data Center TCP (DCTCP): TCP Congestion Control for Data Centers*. RFC 8257. RFC Editor, 2017.
- [12] Mohammad Alizadeh et al. “Data Center TCP (DCTCP)”. In: *Proceedings of the ACM SIGCOMM 2010 Conference*. 2010, pp. 63–74.
- [13] Injong Rhee et al. *CUBIC for Fast Long-Distance Networks*. RFC 8312. RFC Editor, 2018.
- [14] Jana Iyengar and Martin Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. RFC Editor, 2021.
- [15] Bob Briscoe et al. *Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture*. RFC 9330. RFC Editor, 2023.
- [16] Koen De Schepper and Bob Briscoe. *The Explicit Congestion Notification (ECN) Protocol for Low Latency, Low Loss, and Scalable Throughput (L4S)*. RFC 9331. RFC Editor, 2023.
- [17] Koen De Schepper, Bob Briscoe, and Greg White. *Dual-Queue Coupled Active Queue Management (AQM) for Low Latency, Low Loss, and Scalable Throughput (L4S)*. RFC 9332. RFC Editor, 2023.
- [18] Koen De Schepper et al. *Dual Queue Coupled AQM: Deployable Very Low Queuing Delay for All*. arXiv:2209.01078. 2022.
- [19] Maria Eduarda Veras et al. *Implementing the L4S Architecture in the ns-3 Simulator*. arXiv:2603.20166. 2026.
- [20] Yufan Zhuang et al. *OCC: Physical-Layer Assisted Congestion Control for Real-Time Communications*. arXiv:2604.22383. 2026.
- [21] Michael Luby. *ARC: Consistent, Low-Latency Delivery via Receiver-Side Scheduling*. arXiv:2511.16902. 2025.
- [22] Yu Mi et al. “Tree Embedding Based Mapping System for Low-Latency Mobile Applications in Multi-Access Networks”. In: *Proceedings of the IEEE INFOCOM 2025 Conference on Computer Communications*. 2025.
- [23] Gabriel Almeida et al. *Toward Scalable VR-Cloud Gaming: An Attention-Aware Adaptive Resource Allocation Framework for 6G Networks*. arXiv:2512.11667. 2025.
- [24] Ze Wu et al. *EyeNexus: Adaptive Gaze-Driven Quality and Bitrate Streaming for Seamless VR Cloud Gaming Experiences*. arXiv:2509.11807. 2025.
- [25] Hugo Bertin, Marc Dacier, and Yérom-David Bromberg. *Cheating in Multiplayer Online Games: A Dataset*. arXiv:2606.06013. 2026.
- [26] Sally Floyd. *Congestion Control Principles*. BCP 41. RFC Editor, 2000.